



Vulnerability Assessment & Penetration Testing Report

Target:

VulnShop Android Application (APK)

Prepared For:

Hackware Security Solutions LLP

Prepared By:

Basith Mohammed Ali

Date:

09 July 2026

Version:

1.0

Table of Contents

1. Executive Summary.....	3
2. Scope.....	3
3. Methodology.....	3
4. Findings Summary.....	4
F-01 – Insecure Storage of Sensitive Information in SharedPreferences.....	5
F-02 – Insecure Storage of Session Token in SharedPreferences.....	7
F-03 - Authentication Bypass Due to Improper Credential Validation.....	9
F-04 – Hardcoded Sensitive Information in Application Source Cod.....	11
F-05 – Root Detection Bypass.....	13
F-06 – Application Built in Debug Mode.....	15
F-07 – Android Backup Enabled.....	17
Conclusion.....	19

1. Executive Summary

A security assessment was conducted on the **VulnShop Android application** to identify security weaknesses. The assessment included static analysis, dynamic analysis, and manual testing. A total of **7 vulnerabilities** were identified affecting authentication, sensitive data storage, application configuration, and client-side security. The detailed findings, impact, and remediation recommendations are provided in this report.

2. Scope

Target	VulnShop Android Application (APK)
--------	------------------------------------

The assessment focused on authentication, secure storage, application configuration, and client-side security.

3. Methodology

Phase	Description
Static Analysis	Decompiled the APK using JADX to identify security weaknesses.
Dynamic Analysis	Verified vulnerabilities using ADB, Frida, and manual testing.
Vulnerability Validation	Confirmed identified vulnerabilities to eliminate false positives.
Reporting	Documented findings, impact, and remediation recommendations.

4. Findings Summary

ID	Finding	Severity
F-01	Insecure Storage of Sensitive Information in SharedPreferences	High
F-02	Insecure Storage of Session Token in SharedPreferences	Medium
F-03	Authentication Bypass Using Incorrect Password	Critical
F-04	Hardcoded Sensitive Information in Application Source Code	Medium
F-05	Root Detection Bypass	Medium
F-06	Application Built in Debug Mode	Medium
F-07	Android Backup Enabled	Medium

Severity Summary

Severity	Count
Critical	1
High	1
Medium	5
Low	0

F-01 – Insecure Storage of Sensitive Information in SharedPreferences

Severity	High
CVSS v3.1	7.5
Affected Component	SharedPreferences (vuln_prefs.xml)

Description

The Android application stores sensitive authentication information, including the username, password, authentication token, and user information, in plaintext within the application's SharedPreferences file (`vuln_prefs.xml`). The stored values are not encrypted or protected using Android Keystore or EncryptedSharedPreferences.

Static analysis of the application revealed that the `AuthManager` class stores credentials directly in SharedPreferences using the `saveCredentialsInsecurely()` method. Dynamic verification confirmed that the credentials and JWT token could be successfully recovered from the application's private storage.

Proof of Concept

Step 1 – Review the Authentication Code

Using JADX, the `AuthManager` class was analyzed.

Source Code

```
com → vulnshop → demo → data → AuthManager
```

The following method stores user credentials directly into SharedPreferences.

```
public final void saveCredentialsInsecurely(String user, String pass) {  
    this.prefs.edit()  
        .putString("saved_username", user)  
        .putString("saved_password", pass)  
        .apply();  
}
```

```

62     }
63 }
64
65 public final void saveCredentialsInsecurely(String user, String pass) {
66     Intrinsics.checkNotNullParameter(user, "user");
67     Intrinsics.checkNotNullParameter(pass, "pass");
68     VulnerabilityLogger.INSTANCE.logSensitive("AuthManager", "Saving creds for " + user + ": " + pass);
69     SharedPreferences.Editor $this$saveCredentialsInsecurely_u24lambda_u240 = this.prefs.edit();
70     $this$saveCredentialsInsecurely_u24lambda_u240.putString("saved_username", user);
71     $this$saveCredentialsInsecurely_u24lambda_u240.putString("saved_password", pass);
72     $this$saveCredentialsInsecurely_u24lambda_u240.apply();
73 }
74
75 public final Object register(User user, Continuation<? super Unit> continuation) {

```

Step 2 – Login to the Application

Login using a valid account.

Username	basith
Password	Password1

Step 3 – Access Application Storage

Execute

adb shell	run-as com.vulnshop.demo	ls shared_prefs	cat shared_prefs/vuln_prefs.xml
-----------	--------------------------	-----------------	---------------------------------

Step 4 – Verify Sensitive Information

The SharedPreferences file contains sensitive information stored in plaintext.

```

vbox86p:/data/user/0/com.vulnshop.demo $ cat shared_prefs/vuln_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="saved_password">Password1</string>
  <string name="user_token">eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoic3ViIiwiaWF0IjoxNTE2MjM5MDIyfQ.vulnshop123secret</string>
  <string name="saved_username">basith</string>
  <int name="user_id" value="1" />
  <string name="user_name">basith</string>
</map>
vbox86p:/data/user/0/com.vulnshop.demo $

```

saved_username	basith
saved_password	Password1
user_token	eyJhbGc...
user_id	1

Impact

An attacker who gains access to the application's private storage through device compromise, debugging, backup extraction, or other local attack vectors can recover stored credentials and authentication tokens. This may result in unauthorized account access, session hijacking, user impersonation, and compromise of sensitive user information.

Recommendation

Never store passwords in plaintext.

Use Android Keystore or EncryptedSharedPreferences for sensitive data.

Store only short-lived session tokens when required.

Remove authentication data immediately after logout.

Avoid logging sensitive credentials.

F-02 – Insecure Storage of Session Token in SharedPreferences

Severity	Medium
CVSS v3.1	6.5
Affected Component	SessionManager (SharedPreferences – vuln_prefs.xml)

Description

The Android application stores the user's JSON Web Token (JWT) in plaintext within the application's SharedPreferences file (`vuln_prefs.xml`). The session token is not encrypted or protected using Android Keystore or EncryptedSharedPreferences.

Static analysis identified that the `SessionManager` class stores the JWT directly in SharedPreferences using the `saveSessionToken()` method. The application also logs the JWT token through `VulnerabilityLogger`, potentially exposing sensitive session information in application logs. Dynamic verification confirmed that the JWT token could be successfully recovered from the application's private storage.

Proof of Concept

Step 1 – Review the Session Management Code

Using JADX, the `SessionManager` class was analyzed.

Source Code

com → vulnshop → demo → data → SessionManager

The following method stores the session token directly into SharedPreferences.

```
public final void saveSessionToken(String token) {  
    VulnerabilityLogger.INSTANCE.logSensitive(  
        "SessionManager",  
        "Saving JWT token: " + token  
    );  
  
    this.prefs.edit()  
        .putString(Constants.KEY_USER_TOKEN, token)  
        .apply();  
}
```

```
29         this.prefs = sharedPreferences;  
30     }  
31  
32     public final void saveSessionToken(String token) {  
33         Intrinsic.checkNotNullParameter(token, "token");  
34         VulnerabilityLogger.INSTANCE.logSensitive("SessionManager", "Saving JWT token: " + token);  
35         this.prefs.edit().putString(Constants.KEY_USER_TOKEN, token).apply();  
36     }  
37  
38     ...  
39 }
```

Step 2 – Login to the Application

Authenticate using a valid user account.

Step 3 – Access Application Storage

Execute:

<code>adb shell</code>	<code>run-as com.vulnshop.demo</code>	<code>cat shared_prefs/vuln_prefs.xml</code>
------------------------	---------------------------------------	--

Step 4 – Verify the Stored Session Token

The SharedPreferences file contains the JWT session token in plaintext.

```
vbox86p:/data/user/0/com.vulnshop.demo $ cat shared_prefs/vuln_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="saved_password">Password1</string>
  <string name="user_token">eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoic3ViIiwiaWF0IjoxNTE2MjM5MDIyfQ.vulnshop123secret</string>
  <string name="saved_username">basith</string>
  <int name="user_id" value="1" />
  <string name="user_name">basith</string>
</map>
vbox86p:/data/user/0/com.vulnshop.demo $
```

Step 4 – Verify Sensitive Information

The SharedPreferences file contains sensitive information stored in plaintext

Impact

An attacker who gains access to the application's private storage through device compromise, debugging, backup extraction, or other local attack vectors may recover valid session tokens. If the token remains valid, the attacker could hijack authenticated sessions, impersonate legitimate users, and gain unauthorized access to protected application functionality.

Recommendation

- Store session tokens using Android Keystore or EncryptedSharedPreferences.
- Encrypt sensitive session information before storing it locally.
- Avoid storing long-lived session tokens on the device.
- Remove or invalidate session tokens immediately after logout.
- Do not log authentication tokens or other sensitive information in application logs.

F-03 - Authentication Bypass Due to Improper Credential Validation

Severity	Critical
CVSS v3.1	9.1
Affected Component	Login Authentication

Description

The application fails to properly validate user credentials during authentication. During security testing, a valid user account was successfully authenticated using an incorrect password. The application granted access to the authenticated area without verifying the supplied password, resulting in an authentication bypass.

Proof of Concept

Step 1 – Open the Login Screen

Launch the application and navigate to the login page.

Step 2 – Submit Invalid Credentials

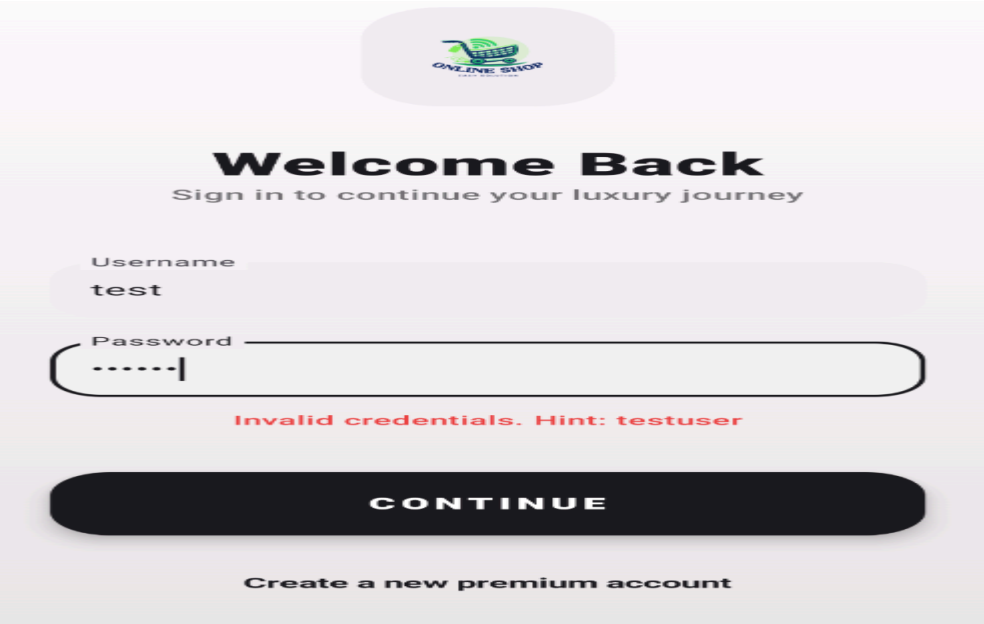
Enter invalid credentials.

Username	test
Password	abc123

Click **Continue**.

The application responds with:

Invalid credentials. Hint: testuser



The screenshot shows a login interface for an 'ONLINE SHOP'. At the top, there is a logo with a shopping cart and the text 'ONLINE SHOP'. Below the logo, the heading 'Welcome Back' is displayed in large, bold letters, followed by the subtitle 'Sign in to continue your luxury journey'. The login form consists of two input fields: 'Username' with the value 'test' and 'Password' with masked characters '.....'. Below the password field, a red error message reads 'Invalid credentials. Hint: testuser'. At the bottom of the form, there is a large black button labeled 'CONTINUE' and a link that says 'Create a new premium account'.

Step 3 – Login Using an Incorrect Password

Enter the following credentials.

Username	testuser
Password	hello123

Note: **hello123** is an intentionally incorrect password.

Click **Continue**.

Step 4 – Verify Authentication Bypass

The application successfully authenticates the user and redirects to the **Discover** page despite the incorrect password.

Impact

An attacker can gain unauthorized access to user accounts without knowing the correct password. This may allow unauthorized users to access protected application features, impersonate legitimate users, and compromise sensitive information.

Recommendation

Validate both the username and password before granting access.
Ensure authentication is performed securely on the server side.
Reject all authentication attempts with invalid credentials.
Remove unnecessary authentication hints from login responses.
Implement account lockout and rate limiting to mitigate authentication attacks.

F-04 – Hardcoded Sensitive Information in Application Source Code

Severity	Medium
CVSS v3.1	6.5
Affected Component	Constants.kt

Description

The application contains multiple sensitive secrets hardcoded within the application source code. Static analysis using JADX identified that the JWT secret, Stripe API key, and Firebase API key are embedded directly inside the **Constants** class.

Since Android APK files can be easily decompiled, an attacker can recover these hardcoded values without requiring authentication. Exposed secrets may assist attackers in further attacks against the application or its backend services.

Proof of Concept

Step 1 – Decompile the APK

Open the application using **JADX**.

Step 2 – Navigate to the Constants Class

Source Code

com	vulnshop	demo	util	Constants
-----	----------	------	------	-----------

Step 3 – Identify Hardcoded Secrets

The following sensitive values were found embedded in the application source code.

```
Constants.kt x
```

```
1 package com.vulnshop.demo.util;
2
3 import kotlin.Metadata;
4
5 /* JADX INFO: compiled from: Constants.Kt */
6 /* JADX INFO: loaded from: classes.dex */
7 @Metadata(d1 = {"\u0000\u0014\n\u00002\u00018\u00002\u00018\u00002\u00018\u00002\b\u00003\u00002\u00018\u0000e\n\u00002\b\u00000b\bC'\u0002\u00018\u000002\u000020\u00018't(\b\u00002$ \u0000\u0004\b\u00002\u0000:\u0000"}, version = 1, kotlin_version = 1.9.0)
8 public final class Constants {
9     public static final int $stable = 0;
10    public static final String APP_NAME = "VulnShop";
11    public static final String BASE_URL = "http://api.vulnshop.demo/";
12    public static final String FIREBASE_API_KEY = "AIzaSyAs-GS_8xZ9_L3_Ml_P0_Q_R";
13    public static final Constants INSTANCE = new Constants();
14    public static final String JWT_SECRET = "vulnshop123secret";
15    public static final String KEY_IS_LOGGED_IN = "is_logged_in";
16    public static final String KEY_USER_DATA = "user_data_json";
17    public static final String KEY_USER_ID = "user_id";
18    public static final String KEY_USER_NAME = "user_name";
19    public static final String KEY_USER_TOKEN = "user_token";
20    public static final String SHARED_PREFS_NAME = "vuln_prefs";
21    public static final String STRIPE_API_KEY = "sk_test_51HzSZJF8XivV7N1V7N1V7N";
22
23    private Constants() {}
24 }
25 }
```

Code Small Simple Fallback ☐ Split view

```
public static final String JWT_SECRET = "vulnshop123secret";
```

```
public static final String STRIPE_API_KEY =
"sk_test_51MzS2ZJF0Xv1vV7N1V7N1V7N";
```

```
public static final String FIREBASE_API_KEY =
    "AlzaSyAs-G5_8xZ9_L3_M1_P0_Q_R";
```

Impact

An attacker can decompile the APK and recover sensitive secrets embedded within the application. Depending on how these secrets are used, they may facilitate unauthorized access, abuse of third-party services, token forgery, or further attacks against the application's backend infrastructure.

Recommendation

Do not hardcode sensitive secrets inside the application.

Store secrets securely on the server.

Use secure key management solutions.

Rotate exposed keys and secrets immediately.

Only include non-sensitive configuration values within the client application.

F-05 – Root Detection Bypass

Severity	Medium
CVSS v3.1	5.3
Affected Component	VulnShopApplication.isRooted()

Description

The application implements root detection by checking for the presence of common root-related files during application startup. However, the root detection mechanism relies entirely on client-side checks and can be bypassed using runtime instrumentation tools such as Frida.

During testing, the `isRooted()` method was successfully bypassed, causing the application to identify a rooted device as non-rooted and continue normal execution.

Proof of Concept

Step 1 – Review the Root Detection Code

Using **JADX**, the application was analyzed.

Source Code

com	vulnshop	demo	VulnShopApplication
-----	----------	------	---------------------

The application checks for common root-related files.

```
private final boolean isRooted() {  
    String[] paths = {  
        "/system/bin/su",
```

```

        "/system/xbin/su",
        "/sbin/su",
        ...
    };

    for (String path : paths) {
        if (new File(path).exists()) {
            return true;
        }
    }

    return false;
}

```

Step 2 – Verify Root Detection

Run the application on a rooted Genymotion emulator.

The application detects the rooted device.

DEVICE IS ROOTED! Security risk detected.

```

07-11 12:32:34.709 2093 2093 D VulnShop_Security: Fetching profile for userId: 999 (VULNERABILITY: IDOR check)
07-11 12:46:48.624 2920 2920 D VulnShop_Security: VulnShop Application initialized
07-11 12:46:48.624 2920 2920 E VulnShop_Security: DEVICE IS ROOTED! Security risk detected.
07-11 12:46:48.671 2920 2920 D VulnShop_Security: MainActivity created. Session valid: false

```

Step 3 – Bypass Root Detection

Start **Frida Server** on the rooted device and execute the root detection bypass script.

```

(kali@kali)-[~/Downloads]
$ frida -U -f com.vulnshop.demo -l root_bypass.js

----
| _ |   Frida 17.9.8 - A world-class dynamic instrumentation toolkit
| ( |
> _ |   Commands:
/_/_|_ |   help      -> Displays the help system
. . . .   object?    -> Display information about 'object'
. . . .   exit/quit  -> Exit
. . . .
. . . .   More info at https://frida.re/docs/home/
. . . .
. . . .   Connected to Pixel 3 (id=127.0.0.1:6555)
Spawned `com.vulnshop.demo`. Resuming main thread!
[Pixel 3::com.vulnshop.demo ]-> [+] Root detection bypassed!

```

Step 4 – Verify the Bypass

After applying the Frida hook, the application reports:

```
07-11 13:34:40.617 3120 3120 D VulnShop_Security: VulnShop Application initialized
07-11 13:34:40.618 3120 3120 D VulnShop_Security: Device appears secure (no root detected).
07-11 13:34:40.644 3120 3120 D VulnShop_Security: MainActivity created. Session valid: true
```

The application continues to operate normally on the rooted device.

Impact

An attacker with the ability to instrument the application at runtime can bypass the client-side root detection mechanism. This reduces the effectiveness of the application's integrity checks and may facilitate further analysis or manipulation of the application on rooted devices.

Recommendation

Do not rely solely on client-side root detection.

Implement multiple root detection techniques.

Detect runtime instrumentation frameworks such as Frida.

Perform sensitive security checks on the server side whenever possible.

F-06 – Application Built in Debug Mode

Severity	Medium
CVSS v3.1	5.3
Affected Component	AndroidManifest.xml

Description

The Android application is built with the `android:debuggable` attribute enabled. A debuggable application allows debugging tools to attach to the running process, making runtime analysis and reverse engineering significantly easier.

Static analysis of the AndroidManifest.xml and dynamic verification using ADB confirmed that the application is installed as a debuggable build.

Proof of Concept

Step 1 – Review the Android Manifest

Using **JADX**, open:

Source Code

Resources → AndroidManifest.xml

The application is configured as:

```
<application
  android:debuggable="true"
  ...
/>
```

Step 2 – Verify the Debuggable Flag

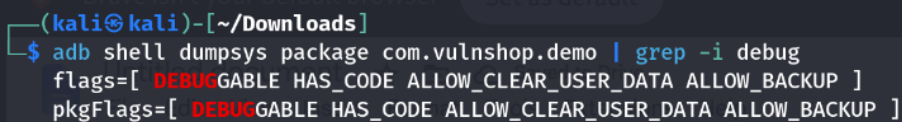
Execute:

```
adb shell dumpsys package com.vulnshop.demo | grep -i debug
```

Output:

```
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]
pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA
ALLOW_BACKUP ]
```

This confirms that the application is installed as a **debuggable build**.



```
(kali@kali)-[~/Downloads]
$ adb shell dumpsys package com.vulnshop.demo | grep -i debug
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]
pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]
```

Impact

An attacker with access to the application can attach a debugger or use runtime analysis tools to inspect application behavior, analyze memory, and facilitate reverse engineering of the application.

Recommendation

Disable the android:debuggable option in production builds.

Use release builds for deployment.

Restrict debugging to development environments only.

F-07 – Android Backup Enabled

Severity	Medium
CVSS v3.1	5.3
Affected Component	AndroidManifest.xml

Description

The Android application is configured with the `android:allowBackup` attribute enabled. This allows the application's data to be included in Android backup mechanisms when supported by the device and platform configuration.

Since the application stores sensitive information locally, enabling application backups increases the risk of unauthorized data extraction from the device.

Static analysis of the AndroidManifest.xml and dynamic verification using ADB confirmed that application backups are enabled.

Proof of Concept

Step 1 – Review the Android Manifest

Using **JADX**, open:

Source Code

Resources → AndroidManifest.xml

The application is configured as:

```
<application
  android:allowBackup="true"
  ...
/>
```

Step 2 – Verify the Backup Configuration

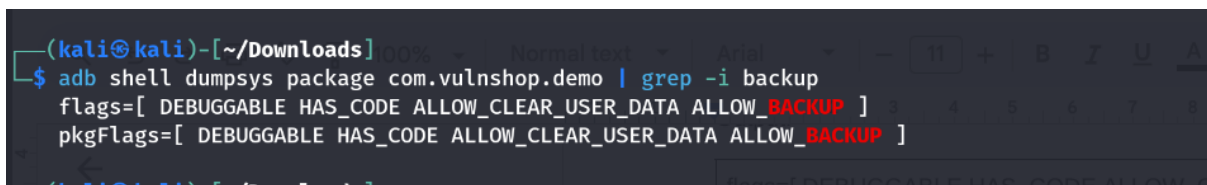
Execute:

```
adb shell dumpsys package com.vulnshop.demo | grep -i backup
```

Output:

```
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]  
pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA  
ALLOW_BACKUP ]
```

The **ALLOW_BACKUP** flag confirms that application backups are enabled.



```
(kali@kali)-[~/Downloads]  
$ adb shell dumpsys package com.vulnshop.demo | grep -i backup  
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]  
pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP ]
```

Impact

If application backups are permitted and sensitive information is stored locally, an attacker with appropriate access to the device or backup data may be able to recover application data, including credentials, session tokens, or other sensitive information.

Recommendation

Disable application backups in production by setting:

```
android:allowBackup="false"
```

Encrypt sensitive application data before storing it locally.

Store sensitive information using secure storage mechanisms such as Android Keystore or EncryptedSharedPreferences.

Conclusion

The security assessment identified multiple vulnerabilities affecting the security of the **VulnShop Android application**, including weaknesses in authentication, sensitive data storage, application configuration, and client-side security. These issues may increase the risk of unauthorized access, sensitive information disclosure, and application compromise. It is recommended that the identified vulnerabilities be remediated and validated through re-testing before deploying the application to a production environment.

End of Report